

Reviewer Pack — Minimal Rank of Hodge Decomposition over Boolean Function Co...

Entry f25d36e23ad6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 18:12:55 UTC

Minimal Rank of Hodge Decomposition over Boolean Function Complexity

Entry ID: f25d36e23ad6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 18:12:55 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Decomposition) **Field B** (complexity object): Complexity Theory: Boolean Function Complexity

Statement:

[‘For every boolean function $f: \{0, \dots, n-1\} \rightarrow \{0, 1\}$, the minimal rank of its associated Hodge decomposition is upper-bounded by a function $\Theta(\log(n))$.’, ‘Equivalently, there exists a constant c such that for all boolean functions f , $\text{rank_Hodge}(f) \leq c \cdot \log(n)$, where $\text{rank_Hodge}(f)$ denotes the minimal dimension over which the Hodge decomposition of f can be represented.’, ‘Furthermore, for any given boolean function g and an error margin $\varepsilon > 0$, there exists a polynomial-time computable algorithm that determines if $\text{rank_Hodge}(g) < c' \cdot \log(n)$ for some constant c' .’]

Rationale (proposer’s reasoning):

[‘The Hodge decomposition is a powerful tool in algebraic geometry that allows for the study of geometric objects by decomposing them into simpler components. Applying this to boolean functions could potentially reveal new structures related to their complexity, providing insights into the nature of computational problems.’, ‘Boolean function complexity measures the difficulty of computing a function with given inputs, which is closely related to complexity theory. By linking Hodge decomposition, an algebraic geometric tool, with boolean function complexity, we may uncover connections between these two fields that could lead to new complexity lower bounds or insights into the structure of computations.’]

Taxonomy category: Minimal Rank (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7a539848106ce495

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 95% of randomly generated n -bit boolean functions have an associated Hodge decomposition rank $\leq c \cdot \log(n) \pm \varepsilon$ for all seeds, where c and ε are constants.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge Decomposition" AND "Boolean Function Complexity" AND minimal rank" - "rank_Hodge" AND boolean function AND upper bound $\Theta(\log(n))$ " - algorithm AND determine rank_Hodge AND polynomial-time computable

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_boolean_function(n):
    return [random.randint(0, 1) for _ in range(2**n)]

def hodge_rank(f):
    n = int(math.log2(len(f)))
    if len(f) != 2**n:
        raise ValueError("Input must be a valid n-bit boolean function")

    count_0 = [0] * n
    count_1 = [0] * n

    for x in f:
        for i in range(n):
            if x & (1 << i) == 0:
                count_0[i] += 1
            else:
                count_1[i] += 1

    rank = max(max(count_0), max(count_1))
    return rank
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        instances_tested = 0
        total_rank = 0

        while instances_tested < 30:
            f = generate_boolean_function(n)
            rank = hodge_rank(f)
            total_rank += rank
            instances_tested += 1

        avg_rank = Fraction(total_rank, instances_tested)
        results.append(avg_rank)

    mean_value = sum(results) / len(results)
    std_dev = math.sqrt(sum((x - mean_value)**2 for x in results) / len(results))

    conjecture_holds = all(rank <= 2 * n_values[i] for i, rank in enumerate(results))
    counterexample = "" if conjecture_holds else f"rank={results[0]}, expected=2*{n_values[0]}"

    return {
        "metric_name": "avg_hodge_rank",
        "metric_value": float(mean_value),
        "instances_tested": len(n_values) * 30,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean_value = sum(results) / len(results)
    std_dev = math.sqrt(sum((x - mean_value)**2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r <= 2 * max(n_values)) / len(results)

    if support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any(r > 2 * max(n_values) for r in results):
        first_failing_seed = seeds[results.index(next(r for r in results if r > 2 * max(n_values)))]
        print(f"RESULT: FALSIFIED counterexample=\"rank exceeds expected\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE support_fraction_too_low")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the allotted time frame. As a result, we cannot confirm or refute the conjecture based on this test. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11314
2	propose	ollama_remote	glm4:latest	0	0	9876
3	propose	ollama_remote	glm4:latest	0	0	10871
4	preregistration	ollama_remote	glm4:latest	0	0	5715
5	novelty	ollama_remote	glm4:latest	0	0	4770
6	novelty	ollama_remote	glm4:latest	0	0	6167
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13993
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10615
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10234
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8892
11	judge	ollama_remote	glm4:latest	0	0	14312

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 106759 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/f25d36e23ad6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f25d36e23ad6.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f25d36e23ad6.tar.gz (if generated)